



daily_stock_analysis 小白指南

LLM 驱动的多市场股票智能分析系统 —— 数据聚合 → 规则指标 → AI 决策仪表盘 → 全渠道推送

GitHub: ZhuLinsen/daily_stock_analysis · MIT License · Python 3.10+ · 实测版本 be148f3 (2026-10-02)

本指南 = 9 个疑问图文解答 + 4 组真实运行的动手实验 (纯技术层面)

①这是什么

②没它之前

③架构

④数据源

⑤指标数学

⑥规则打分

⑦LLM 干啥

⑧防幻觉契约

⑨15种策略

+能力清单+实验

⚠ 免责声明 (必读)：本指南与该项目均**仅供学习研究程序技术**，不构成任何投资建议。AI 生成的评分、买卖点、风险提示**不是投资依据**，作者与本文均不对投资亏损负责。股市有风险，入市需谨慎。本文所有实验只做**代码结构、指标数学与数据契约验证**，全部使用合成数据，不涉及任何真实投资操作。

Q1 这是什么？一句话能说清吗？

一句话：它是一个“每天早上自动给你自选股写研报”的 AI 系统。 你维护一份自选股清单（如 600519, hk00700, AAPL），系统每天定时把每只股票的**行情、K 线、技术指标、新闻、公告、基本面**聚合起来，先做规则化技术分析，再交给大语言模型综合研判，产出一张**“决策仪表盘”：核心结论、0-100 评分、趋势判断、买卖点位、风险警报、催化因素、操作检查清单**，最后推送到企业微信 / 飞书 / Telegram / Discord / Slack / 邮箱。

属性	内容
覆盖市场	A股、港股、美股、日股、韩股、台股 + ETF
AI 模型	Gemini / Claude / OpenAI 兼容 / DeepSeek / 通义 / Ollama 本地模型 (LiteLLM 统一接入)
行情数据	efinance / AkShare / Tushare / Pytdx / Baostock / YFinance / Longbridge / TickFlow
运行方式	GitHub Actions (零成本推荐) / Docker / 本地定时 / FastAPI 服务 / Web 工作台
代码规模	main.py 单文件调度 + src/ 十余个子包 + data_provider/ 19 个数据源适配器 + 500+ 测试文件

Q2 没有它之前人们怎么做？它改变了什么？

以前看一只股票的"日常功课"：开盘前开 3~5 个网站/APP → 看分时行情 → 切到 K 线软件翻 MACD / RSI → 搜新闻公告 → 翻财报摘要 → 脑子里把它们捏成一个结论。**每只股票每天 10~30 分钟，5 只自选股就是一早上**，而且人肉拼接时经常漏掉某条利空公告。

DSA 的改变不是"更聪明的预测"，而是把流水线自动化：



图1：改变的是"信息拼接"这个环节的工作方式，不是预测能力

关键认知：**LLM 在这里做的是"综合研判与表达"，不是"预测涨跌"**——指标和评分由确定性代码算出，LLM 负责把多源信息捏合成有逻辑的结论并诚实标注风险。

Q3 整体架构长什么样？



调度与支撑：main.py (线程池低并发调度，单股失败不影响整体) · src/core/pipeline.py (分析流水线)
config (环境变量全配置化，.env 500+ 行示例) · SQLite 存储 · GitHub Actions / Docker / 本地定时 · 500+ pytest 测试文件
webui.py / server.py: FastAPI Web 工作台 (手动分析/历史报告/回测/持仓/配置管理)

图2：四层管道 —— 数据源 → 规则指标 → LLM 研判 → 多渠道输出，底层是调度与配置

最值得学的工程习惯：**main.py 开头注释就把"交易理念"写明** (严进策略：乖离率>5% 不买；趋势交易：只做 MA5>MA10>MA20 多头排列)，随后所有代码都围绕这几条理念展开——**先写价值观，再写代码**。

Q4 数据从哪来？多数据源 fallback 怎么设计？

免费行情源（AkShare 爬东财等）**随时可能限流、改接口、断网**，所以 DSA 把 19 个 fetcher 做成同接口的适配器，按优先级排队，**第一个成功的当选，失败的自动降级到下一个**——这和知识库里的[[能力层与后端路由]]（每平台有序后端列表，第一个真实可用者当选）是同一个模式在金融数据上的落地。

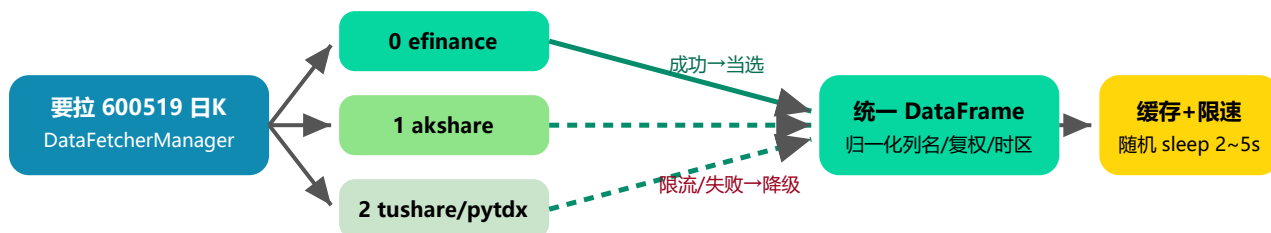


图3：数据源优先级链 + 统一归一化 + 限速防封禁（akshare_fetcher 里带随机 User-Agent 和 sleep）

三个细节值得记住：① **每个 fetcher 都用子进程 + 超时强杀隔离**（`_akshare_call_with_timeout`），防止爬虫库卡死主流程；② 列名归一化（`_normalize_data`）把不同源的中文列名统一成 `open/close/high/low/volume`；③ 新闻是另一条独立数据链（SerpAPI/Tavily/Bocha 等），失败不阻断行情分析，只在报告里如实标注“新闻缺失”。

Q5 技术指标是怎么算的？（可直接抄的公式）

核心在 `src/stock_analyzer.py`，全部用 pandas 十几行搞定，没有任何神秘黑箱：

```
# MA 均线：普通滚动平均
df['MA5'] = df['close'].rolling(5).mean() # MA10/MA20/MA60 同理

# MACD(12,26,9): EMA 是带"记忆衰减"的平均，近期价格权重更大
ema_fast = df['close'].ewm(span=12, adjust=False).mean()
ema_slow = df['close'].ewm(span=26, adjust=False).mean()
df['MACD_DIF'] = ema_fast - ema_slow # 快线
df['MACD_DEA'] = df['MACD_DIF'].ewm(span=9, adjust=False).mean() # 慢线(信号线)
df['MACD_BAR'] = (DIF - DEA) * 2 # 柱状图(红绿柱)

# RSI(6/12/24): Wilder's SMMA 口径（与主流行情软件一致）
gain = delta.where(delta > 0, 0); loss = -delta.where(delta < 0, 0)
avg_gain = gain.ewm(alpha=1/period, adjust=False).mean()
avg_loss = loss.ewm(alpha=1/period, adjust=False).mean()
RSI = 100 - 100 / (1 + avg_gain/avg_loss) # >70 超买, <30 超卖, NaN 填 50(中性)
```

关键口径细节：RSI 用 `ewm(alpha=1/period)`（Wilder 平滑）而不是简单平均——repo 里专门有测试 `test_stock_analyzer_rsi.py` 把 RSI_6/12/24 的期望值精确到小数点后 10 位钉死，还验证了报告模块和告警模块用的是同一公式，防止两处口径漂移。另一个细节：数据不足 60 根时 MA60 直接用 MA20 替代，避免长均线缺数报错。

Q6 规则信号怎么打分？"交易理念"如何变成代码？

作者的交易理念被翻译成 7 步确定性流水线（`analyze()`）：均线计算 → 趋势判断 → 乖离率 → 量能 → 支撑压力 → MACD → RSI → 生成信号。每步产出枚举状态，最后加权合成 **0-100 信号评分 + 六态买入信号（强烈买入/买入/持有/观望/卖出/强烈卖出）**。

理念（main.py 注释）	代码实现	实验实测
严进策略：不追高	乖离率 = $(\text{Close} - \text{MA5}) / \text{MA5}$ ，>5% 不买；强势趋势放宽到 8%	bull 形态乖离 1.8% → "价格贴近 MA5，介入好时机"
趋势交易：只做多头排列	$\text{MA5} > \text{MA10} > \text{MA20}$ 且间距扩大 → STRONG_BULL(90 分)；空头 → 减分	合成空头 K 线 → 空头排列，强度 25
效率优先：筹码集中	量比、缩量回调加权	量比 0.97 → 量能正常
买点偏好：回踩 MA5/MA10	MA5/MA10 支撑判定 + 加分	理由里出现"MA5 支撑有效"

decision-scale-v1：全项目统一的「评分 → 信号 → 动作」契约

80-100 强烈买入 action=buy	60-79 买入 action=buy	40-59 观望 action=watch/hold	20-39 减仓 action=reduce	0-19 卖出 action=sell
----------------------------------	-------------------------------	--------------------------------------	----------------------------------	-------------------------------

规则引擎打分和 LLM 给分都必须落进同一个刻度；不一致时要有 guardrail_reason 说明

图4：decision-scale-v1 五档评分刻度（实验 C 逐档验证通过）

Q7 规则已经打分了，为什么还要 LLM？

规则引擎只看得懂 **数字**（K 线、指标），看不懂 **文本**（"董事长被调查"的公告、突发新闻、行业政策）。LLM 在 DSA 里干三件规则干不了的事：

① **多源综合** 把指标结果 + 10 条新闻 + 公告 + 基本面捏进一个"上下文包"（analysis context pack），让模型通盘权衡；② **表达与解释** 产出人类可读的决策仪表盘：核心结论、催化因素、风险警报、操作检查清单；③ **多轮策略问股**

Web/Bot 里的对话式追问（"帮我用缠论看看这只"），由 `src/agent/` 的 Agent 循环接手——这正是知识库里的 [[Agent 循环]] 的应用：模型点菜、工具执行（`analyze_trend / get_daily_history` 等）、循环到给出答案。

组合方式是典型的 **"确定性代码打底 + LLM 提炼"** 两段式：能算的绝不让模型猜（指标、评分、乖离率全是代码算好的事实），模型只负责事实之上的研判与表达。这比"把行情 JSON 直接扔给 LLM 问涨跌"的粗放做法可靠得多——**事实与判断分离**是这个项目最值得抄的架构决策。

Q8 LLM 输出不可靠怎么办？契约与护栏怎么设计？

金融场景最怕模型“嘴里跑火车”。DSA 的防线一套接一套：

防线	做法
统一评分刻度	decision-scale-v1 (图4)：LLM 给的 sentiment_score 必须落进五档之一，代码用 <code>decision_band_for_score()</code> 强制归一，越界分直接丢弃 (normalize_score 返回 None)
口径一致性守卫	score≥60 但动作却是 hold/watch？必须在 guardrail_reason 写明降级原因，否则提取 DecisionSignal 时被强制对齐
JSON 修复	模型输出的坏 JSON 用 <code>json-repair</code> 库自动修复后再解析
失败不阻断	决策信号写入失败、新闻缺失都不影响主报告——主流程永远先保住“能交付”
可后验评估	每个建议沉淀为 DecisionSignal 结构化记录（入场区间/止损/目标价/失效条件/过期时间），事后可统计命中率，evals/ 目录做 Agent 轨迹评估
诚实披露	数据缺失、新闻为空都会在报告里明示（有专门测试 test_notification_empty_news_disclosure.py 盯着这件事）

这套“契约 + 护栏 + 可后验”三件套，适用于任何让 LLM 输出结构化决策的场景（不只是股票），与知识库 [[质检Gate 与自我纠错循环]] 思路同源。

Q9 “15 种策略问股”是怎么回事？

`strategies/` 目录下 15 个 YAML，每一个 = 一份发给 LLM 的“分析师岗位说明书” + 工具清单：均线金叉、缠论、波浪理论、龙头战法、情绪周期、事件驱动、成长质量、预期重估……用户在 Web 工作台问“用缠论分析 600519”，路由层按 aliases 命中策略，把对应 instructions 注入 Agent 的系统提示。

```
# strategies/ma_golden_cross.yaml (实验 D 实测解析)
name: ma_golden_cross           # 机器名
display_name: 均线金叉         # 展示名
category: trend                 # 策略大类
core_rules: [1, 2, 3]          # 关联交易理念条款
required_tools:                 # 声明该策略要用的工具(Agent按需调用)
  - get_daily_history
  - analyze_trend
aliases: ['均线金叉', '金叉']  # 意图路由关键词
market_regimes: [trending_up]  # 适用的市场状态
instructions: |                 # 619 字符的判定标准说明书(发给LLM)
  主信号: MA5 在最近 3 个交易日内上穿 MA10 ...
  量能确认: 金叉日量比 > 1.2 为积极信号 ...
```

精妙之处：YAML 里写的是“判定标准”而不是“答案”——均线金叉的数学判定由 `analyze_trend` 工具（也就是 Q5 的确定性代码）完成，LLM 只按说明书的逻辑组织证据下结论。策略 = 提示词工程 + 工具约束的组合包，这与知识库 [[AgentSkills技能包]] 是同一思想。



这个项目给我什么：能力清单与用途

多源数据聚合模板

19 个适配器 + fallback + 限速 + 子进程超时隔离，任何"聚合多个不稳定 API"的场景都能抄

确定性指标库

MA/MACD/RSI 的 pandas 参考实现 + 口径锁定测试，可直接搬去自己的数据处理项目

LLM 决策报告流水线

事实与判断分离 + 评分契约 + guardrail，任何"AI 生成结构化评估"场景适用

策略即 YAML

提示词说明书 + 工具清单 + 别名路由，可扩展的"专家人格包"模式

零成本部署范式

GitHub Actions 定时跑批 + 多渠道推送，任何日报类任务都能复用

你能用它干什么（学技术视角）：① 读 `data_provider/base.py` 学适配器 + 管理器模式；② 抄 `stock_analyzer.py` 的指标函数进自己的分析脚本；③ Fork 后配 Secrets 体验"GitHub Actions 当免费服务器"；④ 把它的"契约+护栏"套路用到你自己的 LLM 结构化输出项目；⑤ 15 个 YAML 是现成的提示词工程教材。



动手实验：4 步验证（全部真实运行，纯技术，无投资操作）

目标：不配任何 API Key、不联网拉行情，验证这个系统的三样核心资产——指标数学、规则引擎、输出契约。环境：Windows 11 + Python 3.14，全部命令实测通过。

步骤 1：建最小虚拟环境（怎么操作 → 目的 → 检测）

```
cd F:\reminder\daily-stock-analysis
python -m venv exercise\venv
exercise\venv\Scripts\python -m pip install pandas numpy pydantic python-dotenv pyyaml pytest requests
```

目的：repo 完整 requirements 含 akshare/mini-racer 等重依赖，实验只碰指标与契约层，装最小集即可。检测：

```
exercise\venv\Scripts\python -c "import pandas,numpy,pydantic; print('pandas',pandas.__version__)"
```

✅ 预期结果：输出 **pandas 3.0.6**（实测）；**python -c "from src.stock_analyzer import StockTrendAnalyzer"** 输出 import OK。坑：import 链会依次报缺 pydantic → requests → tenacity → fake_useragent，按报错补装即可，这也是理解模块依赖图的免费教材。

步骤 2：跑仓库自带测试，验证指标口径被“钉死”

```
cd F:\reminder\daily-stock-analysis\repo
..\exercise\venv\Scripts\python -m pytest tests/test_stock_analyzer_rsi.py tests/test_stock_analyzer_bia
```

目的：确认 RSI 用 Wilder 平滑口径且期望值精确到小数点后 10 位、乖离率阈值边界行为正确。

✅ 预期结果（实测）：**12 passed, 11 subtests passed in 0.61s**。其中 **test_calculate_rsi_uses_wilder_ema_for_report_periods** 断言 RSI_6=69.01902761094098——口径漂移在这里第一时间爆红。

步骤 3：合成 K 线喂给规则引擎（实验 A+B）

```
cd F:\reminder\daily-stock-analysis\exercise
venv\Scripts\python exp1_indicators.py
```

目的：① 用三段合成 K 线（多头上升/空头下跌/冲高回落）验证规则引擎能正确区分形态；② 用独立手写的参考实现逐值对比 MA/MACD/RSI，验证指标数学。

✅ 预期结果（实测）：bull → **强势多头/强度90/评分78/买入**；bear → **空头排列**；top → **强势空头/强烈卖出/38分**，形态区分 PASS；参考实现 vs repo 实现 MA5/DIF/DEA/BAR/RSI12 的 |差| 全部 = 0.00e+00，指标口径交叉验证 PASS。改 **make_kline** 里的 seed 或 drift 再跑，观察评分变化——这就是“改参数看变化”的中间成果。

步骤 4：验证评分契约与策略 YAML (实验 C+D)

同一脚本的后半段：把 95/70/50/30/5 五个分数喂给 `decision_band_for_score()`，并解析 `strategies/ma_golden_cross.yaml`。

✅ 预期结果 (实测)：score=95→strong_buy/buy, 70→buy, 50→watch/hold, 30→reduce/sell, 5→sell, 五档映射与文档完全一致；YAML 解析出 `core_rules=[1,2,3]`、`required_tools=[get_daily_history, analyze_trend]`、`aliases`、`instructions` 619 字符；`strategies` 目录 YAML 总数 = 15。



它和量化回测框架 (如 nautilus-trader) 的边界

别把 DSA 当成量化交易系统：**DSA 是“每日研究助理”——生成建议与解释，不执行交易、不做策略回测优化**；
nautilus-trader 那类回测/实盘框架才是“策略引擎”——历史数据回放、撮合模拟、绩效归因。两者的交汇点是 DSA 的 DecisionSignal 资产（结构化建议记录）和内置的轻量回测模块——若要把 AI 建议变成可回测的策略，信号得先经过契约化（DSA 已做）再到回测框架验证（需另接）。一个是**参谋**，一个是**演武场**。